EduAirControl: Plataforma de Microservicios con IoT para el Monitoreo y Análisis Ambiental de Espacios Educativos

Jhon Jairo Cortez Vargas · María de los Ángeles Olaya García
Tecnólogo en Análisis y Desarrollo de Software, Ficha 3145556
Servicio Nacional de Aprendizaje (SENA) Industria, Neiva, Huila, Colombia
Instructora: Karol Daniela Correa

Resumen

Las instituciones educativas rara vez cuentan con un lugar único donde consultar las condiciones ambientales de sus aulas: temperatura, humedad, calidad del aire y ruido suelen medirse de forma aislada o no medirse. Este trabajo describe EduAirControl, una plataforma web y móvil que centraliza esas mediciones, las asocia con el ambiente educativo y el sensor que las produjo, y permite consultarlas en tiempo real y analizarlas por día, semana, mes y año. El sistema se construyó con siete módulos de backend (un API Gateway y seis microservicios) en Java 17 y Spring Boot 4, con autenticación OAuth2/JWT, persistencia en PostgreSQL gestionada con Liquibase, mensajería con RabbitMQ, caché con Redis, un frontend en React y una aplicación móvil en Expo. Un prototipo con ESP32, DHT11, KY-038 y MQ-135 valida la captura física. La calidad se abordó con un plan de pruebas de ocho niveles, 48 casos funcionales y 285 métodos de prueba automatizada. Los resultados que se reportan son los que existen al cierre: la ejecución completa del plan aún no se ha cerrado, el firmware definitivo de envío y la calibración del MQ-135 están pendientes, y una auditoría del código dejó 11 riesgos activos, dos de ellos críticos. El aporte principal es una arquitectura de referencia documentada y verificable para monitoreo ambiental en entornos educativos.

Palabras clave: monitoreo ambiental, Internet de las cosas, microservicios, arquitectura hexagonal, Spring Boot, ESP32, espacios educativos

Introducción

El Problema

Las condiciones ambientales de un aula influyen en el confort y el desempeño de quienes la usan. La revisión de Mendell y Heath (2005) relaciona los contaminantes del aire interior y las condiciones térmicas de las escuelas con el rendimiento estudiantil, y Shield y Dockrell (2003) documentan los efectos del ruido sobre los niños en el contexto escolar. Aun así, la información que producen los sensores suele quedar dispersa: cada dispositivo muestra su propio valor, nadie conserva el historial y no hay una relación clara entre una medición y el espacio donde se tomó.

Quien administra los espacios no puede comparar cómo cambian las condiciones con el tiempo ni decidir con datos. De allí surgen cinco dimensiones del problema: acceso (no hay un lugar claro para consultar la información), centralización (las mediciones quedan aisladas del espacio), comprensión (los valores crudos no explican la evolución), trazabilidad (una medición solo vale si se conocen su origen y su hora) y gobierno (debe controlarse quién ve y quién administra).

Motivación de la Investigación

La motivación fue construir un sistema que no solo funcionara, sino que fuera comprensible, medible y mantenible, y que sirviera como caso de estudio de un proyecto formativo del SENA en el que la documentación, las decisiones de diseño y las pruebas son tan importantes como el código. Además, se quiso comprobar que un prototipo IoT de bajo costo puede integrarse con una arquitectura de microservicios de calidad profesional.

Objetivos de la Investigación

El objetivo general fue desarrollar una plataforma que permita monitorear, gestionar y analizar las condiciones ambientales de espacios educativos mediante datos de sensores IoT. Los objetivos específicos fueron:

- Implementar autenticación y autorización con cuatro roles (SUPER_ADMIN, ADMIN, USER y VIEWER).
- Gestionar campus, ambientes educativos, sensores y dispositivos.
- Recibir, validar, almacenar y analizar mediciones, y generar alertas a partir de umbrales.
- Ofrecer interfaces web y móvil con panel, favoritos, preferencias, idiomas y accesibilidad.
- Validar la captura física de datos con un prototipo basado en ESP32.
- Documentar y probar el sistema de modo que sus resultados sean verificables.

Importancia del Tema

El Internet de las cosas permite conectar objetos físicos con servicios de software para observar su entorno de manera continua (Atzori et al., 2010; Gubbi et al., 2013). En los espacios educativos, esa capacidad habilita decisiones basadas en datos sobre ventilación, ruido o uso de los ambientes. EduAirControl es un caso de aplicación práctica en un contexto colombiano. El producto no promete control físico automático ni gestión predictiva; esas capacidades quedan fuera del alcance actual y se plantean como trabajo futuro.

Trabajos Relacionados

Esta sección no es una revisión sistemática: reúne los referentes que sustentan las decisiones técnicas del proyecto en cuatro frentes.

Internet de las Cosas y Monitoreo Ambiental

Atzori et al. (2010) describen el Internet de las cosas como la convergencia de visiones centradas en los objetos, en el internet y en la semántica, y Gubbi et al. (2013) proponen una arquitectura de referencia donde los sensores alimentan servicios de almacenamiento, análisis y visualización. EduAirControl adopta esa idea: el sensor captura, el backend valida y almacena, y las interfaces presentan.

Microservicios y Arquitectura Hexagonal

Fowler y Lewis (2014) definen los microservicios como pequeños servicios independientes que se comunican mediante mecanismos ligeros y se despliegan por separado. Newman (2015) y Richardson (2018) describen sus compromisos, entre ellos la consistencia de datos y la comunicación asíncrona. Para organizar el interior de cada servicio se usó la arquitectura hexagonal (Cockburn, 2005), que separa el dominio de los adaptadores de entrada y salida, en línea con los principios del diseño guiado por el dominio (Evans, 2003).

Seguridad en APIs

El marco OAuth 2.0 (Hardt, 2012) y los tokens JWT (Jones et al., 2015) son la base de la autenticación del sistema. La lista OWASP Top 10 (OWASP Foundation, 2021) se usó como referencia para revisar controles de acceso, configuración de seguridad y exposición de secretos.

Calidad del Software y Pruebas

La norma ISO/IEC 25010 (International Organization for Standardization y International Electrotechnical Commission [ISO/IEC], 2011) define características de calidad como fiabilidad, seguridad y mantenibilidad. Martin (2008) sostiene que el código limpio reduce el costo de mantenimiento, y Forsgren et al. (2018) vinculan las prácticas de entrega continua con el desempeño de los equipos. Nygard (2018) presenta el patrón de cortacircuitos para limitar fallos en cascada entre servicios, patrón que el proyecto documentó pero aún no implementó.

Metodología

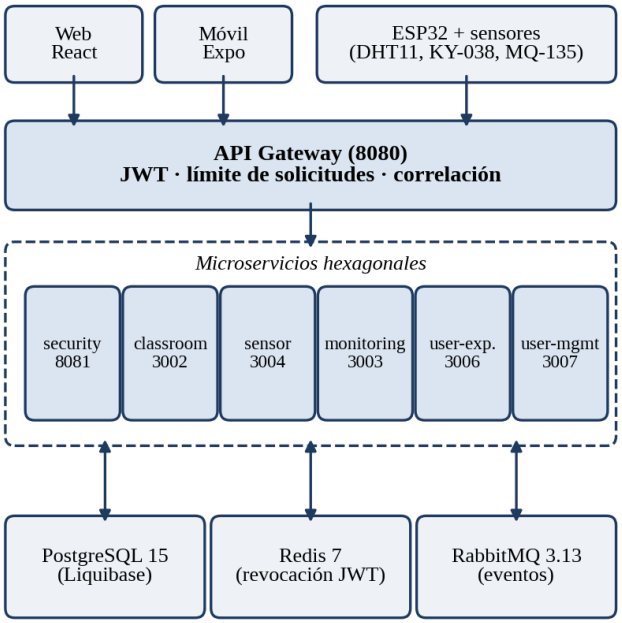
Desarrollo Guiado por Especificación

El equipo trabajó con desarrollo guiado por especificación: primero se documentaron el contexto, el dominio, el producto y los requisitos, y después la arquitectura, los datos y los contratos de API. La documentación se organizó en 15 carpetas de gobierno, contexto, dominio, producto, requisitos, arquitectura, datos, API, UML, microservicios, DevOps, calidad, operación y control del proyecto. Las decisiones de diseño se registraron como 17 registros de decisión de arquitectura (ADR) y los contratos de los servicios se describieron con OpenAPI (OpenAPI Initiative, 2021), con 53 rutas documentadas.

Arquitectura del Sistema

El sistema sigue un modelo cliente–servidor, como muestra la Figura 1. La aplicación web y la móvil consumen una API HTTP/JSON expuesta por un API Gateway (Spring Cloud Gateway, puerto 8080) que enruta, valida el JWT, limita las solicitudes y propaga un identificador de correlación. Detrás trabajan seis microservicios, cada uno con su propia base de datos.

Figura 1
Arquitectura General de EduAirControl



Nota. Los puertos corresponden a la configuración local. Elaboración propia.

Selección de Tecnologías

La Tabla 1 resume la pila tecnológica. Se eligieron tecnologías de uso común en la industria y con documentación abundante, porque el proyecto se desarrolla en un contexto formativo.

Tabla 1
Pila Tecnológica de EduAirControl

Capa	Tecnología
Backend	Java 17, Spring Boot 4.0.5, Spring Data JPA
Seguridad	Spring Authorization Server, JWT RS256, inicio de sesión con Google y Facebook
Datos	PostgreSQL 15, Liquibase
Mensajería y caché	RabbitMQ 3.13, Redis 7
Web	React 19, Vite 8, React Router, i18next, Recharts
Móvil	Expo 54, React Native 0.81, AsyncStorage, BLE PLX
Despliegue	Docker Compose, integración continua con GitHub Actions, manifiestos Kubernetes

Módulos Funcionales

La Tabla 2 presenta los módulos de backend y sus responsabilidades.

Tabla 2
Módulos de Backend

Servicio	Responsabilidad	Puerto
ms-security	Registro, sesiones, OAuth2/OIDC, roles e instituciones	8081
ms-classroom-management	Campus, tipos de ambiente y ambientes	3002
ms-environment-monitoring	Mediciones, panel, análisis, umbrales y alertas	3003
ms-sensor-management	Catálogo, sensores, instalaciones y dispositivos	3004
ms-user-experience	Favoritos, preferencias y calificaciones	3006
ms-user-management	Perfil de usuario	3007

Nota. El API Gateway atiende en el puerto 8080.

Seguridad

El acceso de usuarios se basa en JWT firmados con RS256, con expiración y revocación: al cerrar sesión el token se registra en Redis y su reutilización se rechaza con un código 401. Los servicios validan la firma contra el conjunto de llaves públicas (JWKS) de ms-security. Los cuatro roles separan las operaciones de administración de las de consulta. Los dispositivos usan una clave X-API-Key que se muestra una sola vez al crearse o rotarse. Los secretos se configuran por variables de entorno y CORS es configurable.

Prototipo IoT

El prototipo usa un ESP32 como controlador, un DHT11 para temperatura y humedad, un KY-038 para el nivel relativo de sonido y un MQ-135 para la variación relativa de gases (Espressif Systems, s. f.). El ESP32 se aprovisiona por BLE desde la aplicación móvil. El MQ-135 requiere calibración y adaptación de voltaje antes del conversor analógico, por lo que no se presenta como medidor calibrado de CO_2 en ppm.

Implementación

Estructura de un Microservicio

Cada servicio es un proyecto Maven independiente organizado por contexto, con paquetes de dominio, aplicación e infraestructura. El dominio contiene las reglas (por ejemplo, la evaluación de umbrales y el análisis ambiental), la aplicación orquesta los casos de uso y la infraestructura contiene los controladores, la persistencia y los consumidores de eventos.

Persistencia y Migraciones

Cada servicio mantiene su propio esquema, versionado con Liquibase. Hibernate se configura en modo de validación (ddl-auto=validate), de modo que el esquema solo cambia mediante migraciones y cualquier desviación detiene el arranque.

Mensajería y Consistencia

Los servicios se comunican por eventos en RabbitMQ. Para no perder eventos entre la escritura en la base de datos y la publicación, classroom y las instalaciones de sensores usan el patrón outbox (Richardson, 2018): el evento se guarda en la misma transacción y un proceso programado lo publica. El servicio de monitoreo consume los eventos de los otros servicios.

Flujo de una Medición

El flujo de información ambiental tiene cuatro pasos:

- Los sensores capturan las variables del espacio educativo.
- El ESP32 procesa las lecturas y, en la integración final, las envía al punto de ingesta.
- El backend valida la clave X-API-Key, procesa la medición y la guarda en PostgreSQL; los eventos viajan por RabbitMQ.
- La web y la aplicación móvil consultan la API y presentan mediciones, estados, análisis y alertas.

Despliegue

El entorno local se levanta con Docker Compose y un script de espera que verifica que los servicios estén sanos. Una prueba de humo recorre el flujo de extremo a extremo (registro, inicio de sesión, cierre de sesión y reutilización de un token revocado). Para producción se prepararon manifiestos de Kubernetes con una plantilla de secretos.

Pruebas y Validación

Estrategia de Pruebas

El plan de pruebas (versión 1.0) define ocho niveles: unitarias, integración, contrato, funcionales, sistema, IoT,

regresión y no funcionales. Incluye 48 casos funcionales: 12 de seguridad, 4 de ambientes, 4 de sensores, 7 de monitoreo, 7 de web, 6 de móvil y 8 de IoT. Las técnicas empleadas son partición de equivalencia, valores límite, pruebas negativas, pruebas por rol y pruebas de caja negra. Cada caso termina en uno de cinco resultados (PASS, FAIL, BLOCKED, NOT RUN o NOT APPLICABLE), y una prueba pendiente nunca se registra como aprobada.

Pruebas Automatizadas

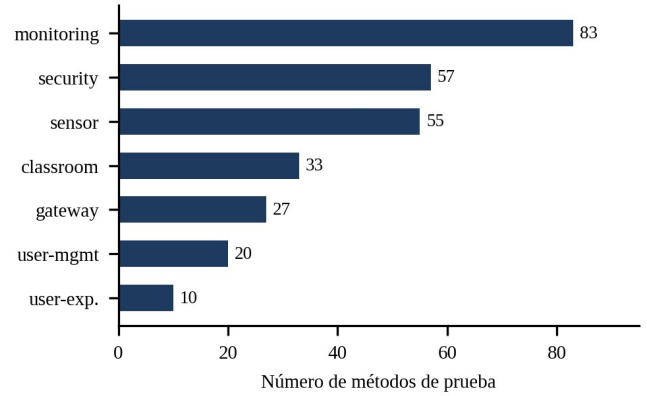
El inventario del código fuente contiene 285 métodos de prueba en 42 clases (Tabla 3 y Figura 2). Según sus anotaciones y clases base, 128 son pruebas unitarias sin contexto de Spring ni contenedores, 89 usan el contexto de Spring (controladores y pruebas de integración) y 68 dependen de contenedores Docker (Testcontainers). Cubren los servicios de aplicación, los controladores, el JWT, la lista de revocación en Redis, la limitación de solicitudes, los consumidores de eventos y las reglas de alertas y análisis.

Tabla 3
Pruebas Automatizadas por Servicio

Servicio	Clases	Métodos
ms-environment-monitoring	12	83
ms-security	8	57
ms-sensor-management	7	55
ms-classroom-management	4	33
API Gateway	6	27
ms-user-management	2	20
ms-user-experience	3	10
Total	**42**	**285**

Nota. Conteo obtenido del código fuente, no de una ejecución.

Figura 2
Métodos de Prueba por Servicio



Nota. Elaboración propia a partir del inventario del código.

Pruebas Manuales

Los 21 casos de web, móvil e IoT son manuales, porque el frontend no tiene pruebas automatizadas. Verifican el inicio de sesión por rol, la navegación del panel, los favoritos, el aprovisionamiento por BLE, la lectura de los sensores y el comportamiento ante fallos (sensor desconectado, permisos revocados, pérdida de Wi-Fi).

Criterios de Aceptación

El plan fija criterios de aceptación para una demostración local (Tabla 4). No son un acuerdo de nivel de servicio productivo.

Tabla 4
Criterios de Aceptación del Plan de Pruebas

Indicador	Criterio	Medición
Verificación de salud	Menos de 1 s	Pendiente
Inicio de sesión	Menos de 2 s	Pendiente
Panel con datos de prueba	Menos de 3 s	Pendiente
Errores HTTP 5xx	0 en el flujo crítico	Pendiente
Disponibilidad	100 % del tiempo planificado	Pendiente

Nota. Pendiente: aún no se ejecutó la medición.

Resultados

Los resultados se presentan tal como existen al 10 de octubre de 2026. No se reportan métricas de desempeño porque todavía no se midieron.

Entregables

Se obtuvo una plataforma con siete módulos de backend, una aplicación web y una aplicación móvil que cubren las ocho capacidades del alcance mínimo: autenticación, ambientes, sensores, monitoreo, panel, análisis histórico, favoritos y calificación de ambientes. Se entregó además la documentación completa (requisitos, análisis, propuesta técnica, diseño, base de datos, plan de pruebas, calidad, implementación y manuales técnico, de instalación, de administración y de usuario) y un prototipo físico que demuestra la lectura de temperatura, humedad y sonido y el aprovisionamiento por BLE.

Estado de la Ejecución de Pruebas

El registro de ejecución del 9 de octubre de 2026 es un borrador: de los 48 casos, 47 están en NOT RUN y uno (IOT-008, envío de mediciones a la API) en BLOCKED, porque no existe el firmware de envío. Como antecedente, los reportes de ejecución del 7 de octubre aparecen en la Tabla 5. En ms-environment-monitoring, 58 de las 86 pruebas fallaron por un bean duplicado en el contexto de Spring; en el código fuente solo existe una clase con ese nombre, pero el directorio de compilación conserva una clase obsoleta, por lo que lo probable es que una compilación limpia lo resuelva. Si el error persiste, se tratará como defecto.

Tabla 5
Reportes de Ejecución Previos (7 de octubre de 2026)

Servicio	Pruebas	Aprobadas	Con error
ms-user-experience	10	10	0
ms-environment-monitoring	86	28	58

Nota. No son el resultado final de la ronda de pruebas.

Registro de Riesgos

La auditoría del código del 8 de octubre de 2026 dejó 11 riesgos activos: 2 críticos, 4 altos y 5 medios. Los críticos son la posibilidad de suplantar identidad mediante las cabeceras X-User-Id y X-User-Role cuando se accede directamente a un servicio, y una llave privada de desarrollo versionada en el repositorio junto con una cuenta superadministradora sembrada con contraseña por defecto. Entre los altos figuran CORS abierto con credenciales, la ausencia de medición de cobertura, un servicio ausente de Kubernetes y de la integración continua, y una observabilidad solo declarada. Otros cuatro riesgos de la misma auditoría ya se cerraron, por ejemplo la incompatibilidad de firmas de tokens entre servicios y un outbox que nunca publicaba.

Discusión

Interpretación

La separación en servicios, los contratos OpenAPI y las migraciones con Liquibase permiten evolucionar cada módulo sin romper a los demás, lo que concuerda con lo planteado por Fowler y Lewis (2014). A su vez, los riesgos cerrados muestran un patrón: las funciones que carecían de prueba de integración (el outbox, la cadena de eventos, las métricas en vivo) fallaban sin que nadie lo notara, lo que respalda la importancia que Martin (2008) y Forsgren et al. (2018) dan a las pruebas y a la entrega continua.

Lecciones Aprendidas

Documentar y auditar con honestidad vale tanto como construir: distinguir lo ejecutado de lo pendiente hace verificables los resultados y convierte las limitaciones en decisiones conscientes. También se aprendió que las decisiones documentadas pueden desviarse de lo desplegado, como ocurrió con la estrategia de base de datos: el ADR planteaba una instancia con un esquema por servicio, mientras que Compose y Kubernetes levantan servidores independientes.

Amenazas a la Validez

El trabajo tiene tres amenazas principales. Primera, los resultados cuantitativos de desempeño y disponibilidad no existen aún, por lo que no se puede afirmar que el sistema cumple los criterios de aceptación. Segunda, el prototipo se probó con lecturas controladas y no con un despliegue real en aulas. Tercera, parte del inventario de pruebas es un conteo estático del código y no garantiza que las pruebas pasen.

Conclusiones

Resumen de Hallazgos

EduAirControl demuestra que es viable centralizar datos ambientales de espacios educativos en una plataforma segura, trazable y escalable, con microservicios hexagonales y un prototipo IoT de bajo costo. Su calidad, sin embargo, aún no está demostrada con mediciones: lo que existe es una base verificable (plan, casos, inventario de pruebas y registro de riesgos) lista para ejecutarse.

Contribuciones

Como producto, se entrega un sistema funcional con web, móvil y prototipo físico. Como aporte técnico, se documenta una arquitectura de referencia con 17 decisiones justificadas y 53 rutas contractualizadas. Como aporte metodológico, se presenta un plan de pruebas con un esquema de resultados que impide registrar como aprobado lo que no se ha ejecutado.

Limitaciones

No existe aún el firmware definitivo que envíe mediciones a la API. El MQ-135 no está calibrado. No hay pruebas automatizadas del frontend. No se hicieron pruebas de carga, de penetración certificada ni de recuperación ante desastres. Los dos riesgos críticos de seguridad siguen abiertos.

Trabajos Futuros

A corto plazo: ejecutar y cerrar el plan de pruebas, corregir los riesgos críticos de seguridad, medir la cobertura con JaCoCo y completar el firmware de ingesta con reconexión Wi-Fi. A mediano plazo: calibrar el MQ-135 o sustituirlo por un sensor de CO_2 certificado, incorporar pruebas automatizadas de web y móvil, implementar el cortacircuitos y la observabilidad, y desplegar en un entorno de pruebas para validar el retroceso de versiones. A largo plazo: evaluar el sistema en aulas reales durante un periodo prolongado y estudiar alertas predictivas.

Reflexión Final

Un sistema medible empieza por decir con claridad qué se ha medido y qué no. Esa es la conclusión que deja este proyecto: la calidad no se declara, se demuestra, y mientras tanto debe quedar escrito qué falta.

Referencias

Atzori, L., Iera, A., y Morabito, G. (2010). The Internet of Things: A survey. *Computer Networks, 54*(15), 2787–2805. https://doi.org/10.1016/j.comnet.2010.05.010

Cockburn, A. (2005). *Hexagonal architecture*. https://alistair.cockburn.us/hexagonal-architecture/

Equipo EduAirControl. (2026a). *Manual técnico de EduAirControl* (Versión 1.1) [Documento técnico]. SENA Industria.

Equipo EduAirControl. (2026b). *Plan de pruebas de EduAirControl* (Versión 1.0) [Documento técnico]. SENA Industria.

Equipo EduAirControl. (2026c). *Registro de riesgos del proyecto* [Documento técnico]. SENA Industria.

Espressif Systems. (s. f.). *ESP32 series datasheet*. https://www.espressif.com/

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Forsgren, N., Humble, J., y Kim, G. (2018). *Accelerate: The science of lean software and DevOps*. IT Revolution Press.

Fowler, M., y Lewis, J. (2014). *Microservices: A definition of this new architectural term*. https://martinfowler.com/articles/microservices.html

Gubbi, J., Buyya, R., Marusic, S., y Palaniswami, M. (2013). Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems, 29*(7), 1645–1660. https://doi.org/10.1016/j.future.2013.01.010

Hardt, D. (Ed.). (2012). *The OAuth 2.0 authorization framework* (RFC 6749). Internet Engineering Task Force. https://doi.org/10.17487/RFC6749

International Organization for Standardization y International Electrotechnical Commission. (2011). *Systems and software engineering—Systems and software quality requirements and evaluation (SQuaRE)—System and software quality models* (ISO/IEC 25010:2011).

Jones, M., Bradley, J., y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.

Mendell, M. J., y Heath, G. A. (2005). Do indoor pollutants and thermal conditions in schools influence student performance? A critical review of the literature. *Indoor Air, 15*(1), 27–52. https://doi.org/10.1111/j.1600-0668.2004.00320.x

Newman, S. (2015). *Building microservices: Designing fine-grained systems*. O'Reilly Media.

Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2.ª ed.). Pragmatic Bookshelf.

OpenAPI Initiative. (2021). *OpenAPI specification* (Versión 3.1.0). https://spec.openapis.org/oas/v3.1.0

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning Publications.

Shield, B. M., y Dockrell, J. E. (2003). The effects of noise on children at school: A review. *Building Acoustics, 10*(2), 97–116.

Apéndice

Datos: las cifras provienen del código fuente del proyecto (inventario de pruebas), de los reportes de ejecución del 7 de octubre de 2026 y del registro de riesgos. No se usaron datos personales reales.

Código: repositorio del proyecto, en el archivo 15_Enlace_Repositorio_GitHub.txt de la entrega; versión (commit) por registrar.

Entorno: Windows; los reportes previos muestran JDK 25.0.2, mientras que los POM declaran Java 17. Versión de Docker y Node por registrar.

Anexos: contratos OpenAPI, ADR, archivos de infraestructura, registro de riesgos, capturas e inventario de pruebas (carpeta 14_Anexos).